

Final Report:

Hardware–Software Co-Design of an FPGA-Accelerated Image Signal Processing Pipeline on a RISC-V SoC Platform

Yicheng Wang

Student ID: 123090586

December 10, 2025

Abstract

Following the completion of Phase 2 of the mid-term project, which established the functional software model of the Image Signal Processing (ISP) pipeline, this report presents the progress of our hardware–software co-design workflow targeting the BeagleV-Fire platform (RISC-V + FPGA).

Phase 3 focuses on constructing a reproducible development environment on the PolarFire SoC and establishing the software performance baseline. A full C++ implementation of the ISP pipeline—comprising demosaic, auto white balance, and 3×3 denoise filtering—was ported and executed natively on the RISC-V CPU. Using both wall-clock timing measurements and `gprof` statistical profiling, we obtained a detailed performance breakdown on real embedded hardware. The results show that the denoise module dominates the total execution time $\approx 60\text{--}66\%$, clearly identifying it as the primary computational bottleneck. This profiling step provides the quantitative basis for hardware acceleration.

Building directly upon the insights from Phase 3, Phase 4 focuses on the theoretical study and architectural design of the FPGA-based denoise accelerator. We investigated the CPU–FPGA interaction model, AXI4-Lite register mapping, and the configuration mechanisms required for accelerator control. Additionally, we analyzed the internal architecture of a high-throughput denoise core, including dual line buffers, a 3×3 sliding-window generator, and a pipelined convolution kernel capable of sustaining a one-pixel-per-cycle throughput. Further study of the AXI4-Stream interface clarified the handshake protocol, backpressure behavior, and frame-completion detection. Finally, we examined the structure of the RISC-V driver software and its role in coordinating register access, pixel streaming, and execution synchronization.

Together, Phase 3 and Phase 4 complete the transition from software profiling to architectural planning, establishing the theoretical and methodological foundation required for the FPGA implementation and system integration in the upcoming development stages.

1 Phase III: Baseline Implementation & Profiling on RISC-V

1.1 Development Environment Setup

To enable efficient hardware–software co-design on the BeagleV-Fire platform, we first established a remote development workflow. The BeagleV-Fire utilizes a Microchip PolarFire SoC with a RISC-V (RV64GC) architecture.

Initial attempts to connect via the standard Visual Studio Code "Remote - SSH" extension were unsuccessful. As shown in Figure 1, the standard VS Code Server does not currently support the RISC-V architecture natively, resulting in a connection failure.



Figure 1: Compatibility issue encountered when attempting to use VS Code Remote-SSH with the RISC-V architecture.

To resolve this, we implemented a hybrid workflow:

III.A.1 SSH Terminal: A standard SSH connection (`ssh beagle@192.168.137.97`) was used for compilation and execution control.

III.A.2 SFTP Synchronization: The VS Code SFTP extension (fork by Natizyskunk) was configured to automatically upload source code upon saving. This allows for local editing with instant remote synchronization to the `/home/beagle/project` directory on the SoC.

1.2 Software-Only Baseline Implementation

A complete Image Signal Processing (ISP) pipeline was implemented in C++17 to serve as the performance baseline. The pipeline consists of three stages:

III.B.1 Demosaic: Bilinear interpolation to convert Bayer Pattern RAW data to RGB.

III.B.2 Auto White Balance (AWB): Gray-world algorithm to correct color temperature.

III.B.3 Denoise: A 3×3 spatial convolution filter (Gaussian approximation).

The software pipeline takes a clean RGB image (`input.jpg`) as input, simulates sensor acquisition via a noisy Bayer RAW pattern, and then reconstructs a denoised RGB image that is written back to disk as `output_result.png`. Figures 2 and 3 show a visual comparison between the input reference frame and the pure software ISP output generated on the RISC-V platform.



Figure 2: Reference RGB input frame (`input4.jpg`) used to generate fake RAW data for the ISP baseline.



Figure 3: ISP output frame (`output_result5.png`) produced by the software pipeline on RISC-V after demosaic, AWB and 3×3 denoise.

The code was compiled directly on the BeagleV-Fire using the GNU Compiler Collection (GCC) with optimization level 2 enabled to ensure a fair comparison against future hardware acceleration.

Listing 1: Compilation command for baseline profiling

```
g++ -O2 -std=c++17 -pg main.cpp -o isp_riscv-pg
```

1.3 Performance Analysis

The software pipeline was executed on a 1200×2130 test image, looping 100 times to gather stable timing data. We utilized two methods for profiling: manual instrumentation using `std::chrono` and statistical profiling using GNU `gprof`.

1.3.1 Wall-Clock Execution Time

Figure 4 shows the runtime execution log. The total system time for 100 frames was approximately 154 seconds. The Denoise stage was significantly slower than other stages.

```

1. Loading input image: input.jpg
   Image loaded: 1200x2130
2. Generating fake RAW data...
3. Running ISP pipeline (looping 100 times)...
   Loop 0/100...
   Loop 10/100...
   Loop 20/100...
   Loop 30/100...
   Loop 40/100...
   Loop 50/100...
   Loop 60/100...
   Loop 70/100...
   Loop 80/100...
   Loop 90/100...

=====
                        PERFORMANCE REPORT (100 FRAMES)
                        Platform: RISC-V (BeagleV-Fire)
=====
Function Name      | Total Time (ms) | Percentage
-----|-----|-----
Pipeline_Demosaic |      24143.63 ms |      15.66%
Pipeline_AWB       |      27991.63 ms |      18.15%
Pipeline_Denoise   |     102051.51 ms |      66.19%
-----|-----|-----
TOTAL SYSTEM TIME |     154186.77 ms |     100.00%
=====
4. Saving result...
Done! Saved to output result.png

```

Figure 4: Terminal output showing execution time distribution across ISP stages on the RISC-V CPU.

1.3.2 Function-Level Profiling (gprof)

To verify the specific bottleneck, we analyzed the execution using **gprof**. The flat profile results are summarized in Table 1.

Table 1: gprof Flat Profile Results (Top 10 Functions)

Function Name	Time (%)	Self (s)	Calls	s/call
Pipeline_Denoise	60.31	94.63	100	0.95
Pipeline_AWB	17.85	28.00	100	0.28
Pipeline_Demosaic	15.39	24.14	100	0.24
stbi_zlib_compress	3.74	5.87	1	5.87
stbiw__encode_png_line	0.87	1.36	12780	0.00
stbi_write_png_to_mem	0.44	0.69	1	0.69
stbiw__zlib_flushf	0.26	0.41	5361406	0.00
stbiw__zhash	0.20	0.31	4431411	0.00
GenerateFakeRaw	0.20	0.31	1	0.31
stbi__YCbCr_to_RGB_row	0.17	0.27	2130	0.00

1.4 Conclusion of Phase III

Both timing methods confirm that the **Denoise Algorithm** is the primary bottleneck, consuming approximately 60% to 66% of the total execution time.

The **Pipeline_Denoise** function relies on nested loops performing convolution on every pixel channel, which is computationally expensive for a general-purpose RISC-V CPU. This confirms that the Denoise module is the ideal candidate for offloading to the FPGA fabric in the subsequent Hardware-Software Co-Design phase.

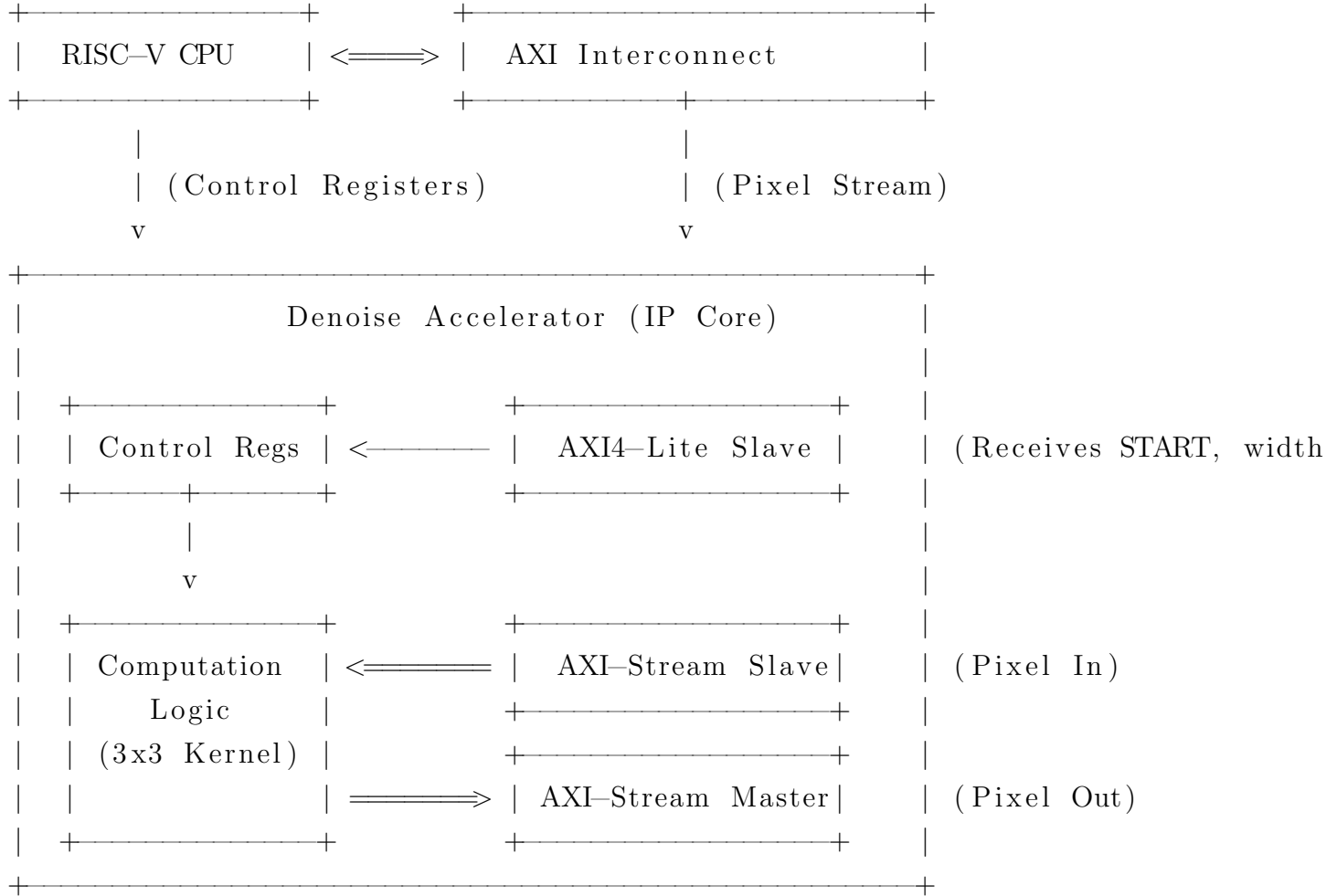
2 Phase IV: Defining Interfaces and Register Specification

Before beginning the practical Verilog implementation, it is essential to first establish the theoretical foundations required for building the FPGA accelerator. Phase IV therefore serves as a consolidated knowledge-preparation stage, summarizing the system-level architecture, interface protocols, and control specifications that guide the later hardware design. This includes understanding the accelerator’s placement within the SoC, defining hardware interfaces such as AXI4-Stream and AXI4-Lite, specifying the memory-mapped register set, analyzing the internal denoise processing core architecture, studying the behavior of the AXI4-Stream wrapper for pixel data flow, and examining the AXI4-Lite control/status module and its FSM. Together, these topics form the theoretical blueprint of the system, ensuring that subsequent implementation work proceeds with clear interface definitions and a well-structured design framework.

2.1 System-Level Block Diagram

The position of the Denoise Accelerator within the SoC system is illustrated below:

Listing 2: System-level block diagram of the Denoise Accelerator



2.2 Hardware Interface Definitions

Two types of interfaces are required:

- **Data Streaming Interface (AXI4-Stream)**
- **Control and Status Interface (AXI4-Lite)**

2.2.1 Data Streaming Interface

The accelerator receives and outputs pixel data through AXI4-Stream. Since image pixels form a continuous stream, this protocol naturally suits line-buffered filtering operations.

Although RGB pixels are 24 bits, the hardware design is simplified by processing each channel separately using an 8-bit data path. This matches the software implementation where R, G, and B channels are processed independently.

Table 2: AXI4-Stream Signals for Pixel Input/Output

Signal	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Active-low reset
s_axis_tdata	Input	8	Input pixel data
s_axis_tvalid	Input	1	Upstream asserts valid data
s_axis_tready	Output	1	Accelerator ready to receive
m_axis_tdata	Output	8	Output pixel data
m_axis_tvalid	Output	1	Output data is valid
m_axis_tready	Input	1	Downstream ready to consume

AXI-Stream Port Definition

2.2.2 Control Interface (AXI4-Lite)

The accelerator exposes a set of memory-mapped registers that allow the CPU to configure parameters and query the execution status. AXI4-Lite is used because the bandwidth requirements are low.

Table 3: Abstract Control Interface Signals

Signal	Direction	Description
reg_addr	Input	Register address
reg_wdata	Input	Write data
reg_rdata	Output	Read data
reg_write	Input	Write enable
reg_read	Input	Read enable

2.3 Register Map Specification

The register map defines how the CPU configures the accelerator. To keep the memory-mapped control interface simple, all registers are defined as 32-bit words and aligned to 4-byte boundaries.

Assume the base address of the accelerator is:

0x4000_0000

Table 4: Register Map of the Denoise Accelerator

Offset	Name	R/W	Description
0x00	CONTROL_REG	R/W	Bit0: START (begin processing) Bit1: AUTO_RESTART (optional)
0x04	STATUS_REG	R/O	Bit0: BUSY (1 = processing) Bit1: DONE (frame completed)
0x08	IMAGE_WIDTH	R/W	Image width in pixels
0x0C	IMAGE_HEIGHT	R/W	Image height in pixels
0x10	THRESHOLD	R/W	Denoising threshold (optional)

Using these registers, the CPU operates the accelerator as follows:

IV.C.1 Write IMAGE_WIDTH and IMAGE_HEIGHT.

IV.C.2 (Optional) Write THRESHOLD.

IV.C.3 Write START = 1 to the CONTROL_REG.

IV.C.4 Poll STATUS_REG.DONE or wait for an interrupt.

IV.C.5 Start a new frame by writing START again, or enable AUTO_RESTART.

2.4 Denoise Processing Core Architecture

The denoise accelerator implements the 3×3 spatial filtering operation identified as the performance bottleneck in the software baseline. To achieve one-pixel-per-cycle throughput, the hardware design adopts a fully pipelined architecture consisting of line buffers, a sliding window generator, and a convolution kernel. This step specifies the datapath, internal buffering strategy, boundary handling method, and timing characteristics.

2.4.1 Overview of the Processing Core

The goal of the processing core is to produce a filtered output pixel for every incoming pixel after the initial pipeline fill latency. The computation follows the standard 3×3 mean (or thresholded mean) filter:

$$D = \frac{1}{9} \sum_{i=1}^9 p_i,$$

where p_i are the pixel values within the 3×3 neighborhood. Since the filter is separable by color channel, the hardware operates on 8-bit grayscale-like data; R, G, and B channels are streamed separately by software.

The accelerator maintains a throughput of:

$$\text{Throughput} = 1 \text{ pixel per cycle}$$

after window initialization.

2.4.2 Sliding Window and Line Buffer Organization

A 3×3 window requires pixels from three consecutive rows. Since input pixels arrive in raster-scan order (left-to-right, top-to-bottom), the accelerator constructs the window using a combination of:

- Two line buffers for storing previous rows,
- Three 3-element shift registers to form the horizontal window.

The structure is illustrated below:

Listing 3: 3×3 sliding window structure

```
[p11] [p12] [p13]    <— Row i-1 (line buffer 1)
[p21] [p22] [p23]    <— Row i   (line buffer 2)
[p31] [p32] [p33]    <— Row i+1 (current input row)
```

Each clock cycle inserts a new pixel at position $p33$, shifting all columns left. The line buffers shift vertically, while local registers shift horizontally.

Line Buffer Characteristics

- Each buffer stores exactly one image row: IMAGE_WIDTH bytes.
- Implemented using BRAM or shift-register LUTs depending on synthesis.
- Provides two outputs per cycle: the element in the current column of row $i - 1$ and row i .

Window Validity Condition The 3×3 window becomes valid only after:

- At least 2 full rows have been received,
- At least 2 pixels have been received in the current row.

Thus the first valid output pixel appears after:

$$\text{Latency} = 2 \times \text{IMAGE_WIDTH} + 2 \text{ cycles.}$$

2.4.3 Convolution Kernel Pipeline

Once the window is valid, the computation proceeds every cycle. The kernel performs:

1. **Summation:** Add the 9 pixel values.
2. **Averaging:** Divide by 9 using multiplication by a pre-computed constant or a shift-and-add method.
3. **Optional thresholding:** Replace p_{22} (center pixel) with the mean if its deviation exceeds a user-defined threshold.

Mathematically, with threshold T :

$$D = \begin{cases} p_{22}, & |p_{22} - \text{avg}| < T \\ \text{avg}, & \text{otherwise.} \end{cases}$$

Pipeline Stages A typical implementation divides the kernel into:

- Stage 1: Partial sums (3 parallel adders).
- Stage 2: Final sum + average computation.
- Stage 3: Threshold comparison and clamping.

This ensures:

$$\text{Throughput} = 1 \text{ pixel/cycle}, \quad \text{Latency} \approx 3 \text{ cycles.}$$

2.4.4 Boundary Handling Strategy

At image boundaries, not all window elements exist. The accelerator applies **replicate padding**:

$$p(x, y) = p(\max(\min(x, W - 1), 0), \max(\min(y, H - 1), 0)).$$

This method:

- Matches the CPU baseline implementation,
- Avoids introducing artificial black edges,
- Requires minimal additional hardware (simple pixel forwarding).

2.4.5 Timing Behavior

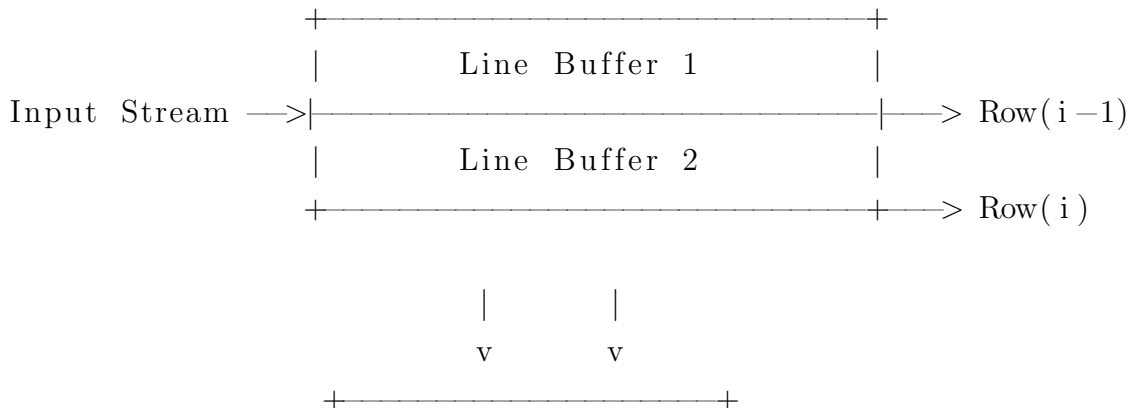
A timing summary:

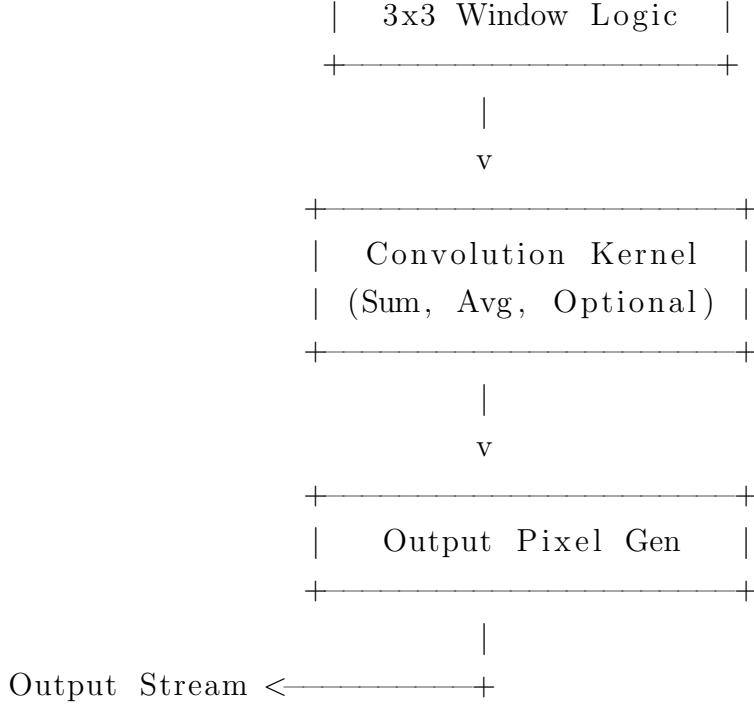
- **Input rate:** 1 pixel per cycle.
- **Output rate:** 1 pixel per cycle (after window warmup).
- **Pipeline latency:** 2 rows + 2 pixels + kernel latency.
- **Total operation cycles:**

$$\text{Cycles} = H \times W + \text{latency}.$$

2.4.6 Internal Core Block Diagram

Listing 4: Internal datapath of the denoise accelerator





The modularity of this architecture allows the computation core to be wrapped by AXI4-Stream logic and controlled by the AXI4-Lite module.

2.5 AXI4-Stream Wrapper Design

The purpose of the AXI4-Stream wrapper is to interface the internal denoise computation core with the external pixel data stream provided by the RISC-V processor or DMA engine. The wrapper ensures correct handshake behavior, manages pipeline warm-up and backpressure, and generates appropriate valid/ready signals for both input and output streams.

AXI4-Stream is chosen because the denoise accelerator consumes and produces a continuous flow of pixel data, and address-based access is unnecessary for convolution-based filters.

2.5.1 AXI4-Stream Protocol Overview

A valid data transfer occurs when both the following conditions are met:

$$s_axis_tvalid = 1 \quad \text{and} \quad s_axis_tready = 1.$$

Similarly, for the output stream:

$$m_axis_tvalid = 1 \quad \text{and} \quad m_axis_tready = 1.$$

This handshake provides robust flow control:

- The accelerator may pause input by deasserting `s_axis_tready`.
- The downstream consumer may pause output by deasserting `m_axis_tready`.

2.5.2 Input Data Handling

The wrapper feeds incoming pixels into the line buffers and sliding window generator.

Input Accept Condition The wrapper asserts `s_axis_tready` when:

- The computation core is enabled (after START),
- The line buffers are not full,
- The accelerator is not in DONE state.

Thus:

$$s_axis_tready = core_ready \wedge \neg done.$$

Each clock cycle satisfying:

$$s_axis_tvalid \wedge s_axis_tready$$

pushes the new pixel into:

- the bottom row of the window,
- the second line buffer (shift),
- the first line buffer (shift).

2.5.3 Window Validity and Compute Enable

The internal computation core may only run when the 3×3 window is valid. The condition is:

$$window_valid = (y \geq 2) \wedge (x \geq 2),$$

where x and y are current pixel coordinates.

For invalid windows (top-left region), output is suppressed and `m_axis_tvalid` remains 0.

2.5.4 Output Data Generation

Once the window becomes valid and the convolution kernel produces a value, the wrapper asserts:

$$m_axis_tvalid = \text{compute_valid}.$$

The downstream consumer (CPU or DMA) controls the acceptance of output pixels via `m_axis_tready`. If `m_axis_tready = 0`, the wrapper must buffer the result until it can be transmitted.

Backpressure Handling Output stalls propagate backward:

1. If the output buffer is full, the computation core stalls.
2. If the computation core stalls, line-buffer shifting stalls.
3. When line buffers stall, `s_axis_tready` is deasserted.

This guarantees lossless streaming even if the downstream pipeline is slower.

2.5.5 Frame Completion Detection

The wrapper monitors pixel coordinates:

$$(x, y) \in [0, W - 1] \times [0, H - 1].$$

Frame completion is detected when the last output pixel is produced:

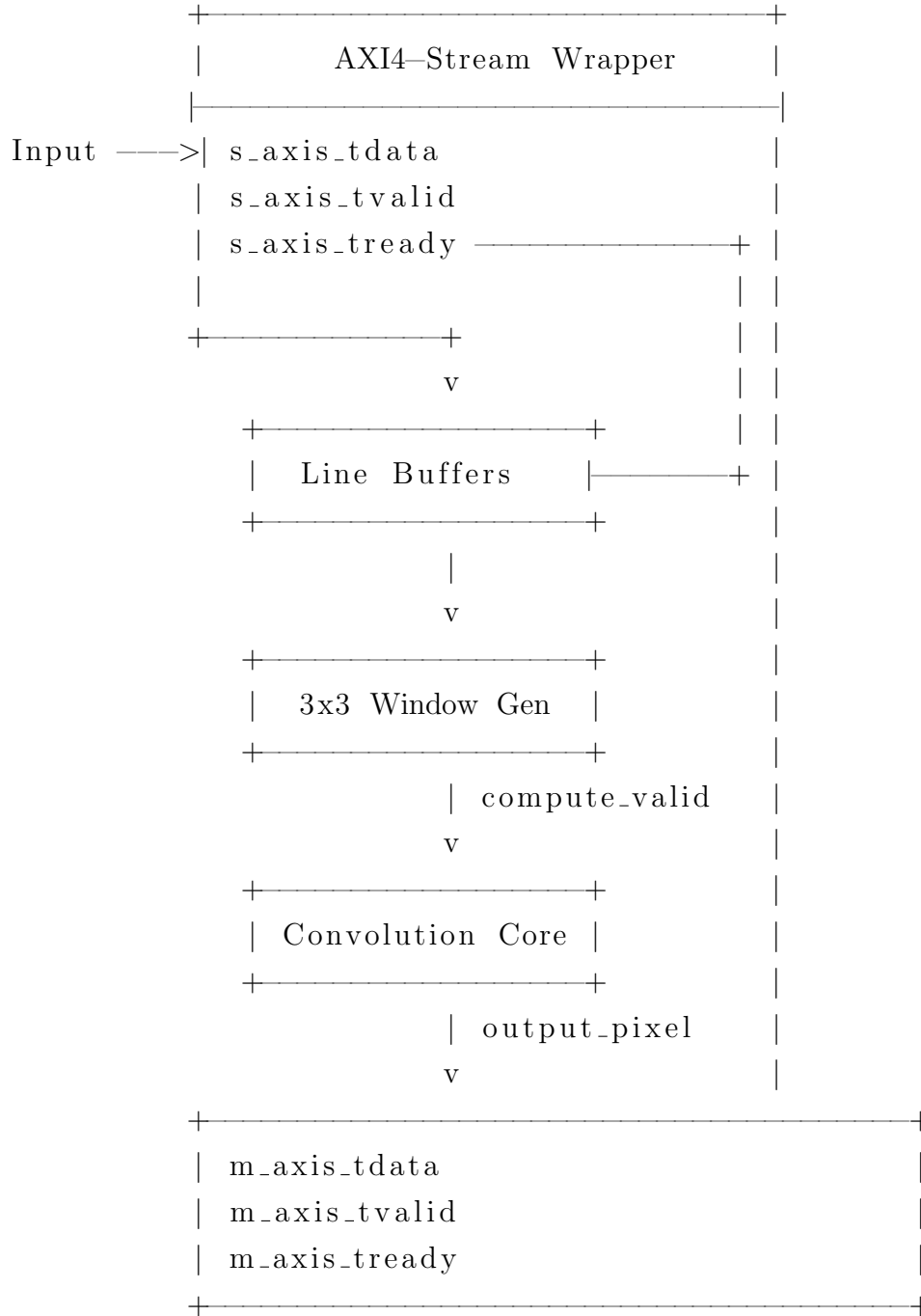
$$(y = H - 1) \wedge (x = W - 1).$$

At that moment:

- The wrapper asserts internal `frame_done`,
- The AXI-Lite control module sets `STATUS_REG.DONE = 1`,
- The FSM transitions to the DONE state.

2.5.6 AXI4-Stream Wrapper Block Diagram

Listing 5: AXI4-Stream wrapper around the denoise core



2.5.7 Summary of AXI4-Stream Behavior

- Input and output each follow AXI4-Stream ready/valid handshake.

- The wrapper hides pipeline warmup and boundary conditions.
- Backpressure propagates cleanly through the entire datapath.
- One-pixel-per-cycle throughput is preserved under continuous streaming.

This wrapper allows the internal computation core to operate seamlessly within the AXI-based SoC architecture and prepares the accelerator for control integration using AXI4-Lite.

2.6 AXI4-Lite Control and Status Module

The AXI4-Lite control module exposes the accelerator as a memory-mapped peripheral to the RISC-V CPU. It implements the register map defined earlier, decodes read/write transactions, and drives the internal control FSM (Finite State Machine) that manages START, BUSY, and DONE behavior.

2.6.1 Register Map Recap

For clarity, the register map is repeated here:

Table 5: Denoise Accelerator Register Map (AXI4-Lite)

Offset	Name	R/W	Description
0x00	CONTROL_REG	R/W	Bit 0: START (write 1 to start) Bit 1: AUTO_RESTART (optional)
0x04	STATUS_REG	R/O	Bit 0: BUSY (1 = processing) Bit 1: DONE (1 = frame complete)
0x08	IMAGE_WIDTH	R/W	Image width in pixels
0x0C	IMAGE_HEIGHT	R/W	Image height in pixels
0x10	THRESHOLD	R/W	Denoise-related parameter (optional)

2.6.2 AXI4-Lite Slave Interface Role

The AXI4-Lite slave interface receives address, data, and control signals from the interconnect and translates them into register-level operations:

- **Write path:** update internal registers and generate side effects (e.g., starting the FSM when START is written).
- **Read path:** return the current register values, including BUSY and DONE flags.

The abstract internal signals used by the core are:

- `cfg_width`, `cfg_height`, `cfg_threshold`
- `ctrl_start`, `ctrl_auto_restart`
- `status_busy`, `status_done`

2.6.3 Control FSM Definition

The control logic is modeled as a simple FSM with four states:

Listing 6: High-level control FSM

IDLE $\longrightarrow$ START $\longrightarrow$ BUSY $\longrightarrow$ DONE $\longrightarrow$ IDLE

IDLE State

- No frame is currently being processed.
- `STATUS_REG.BUSY = 0`, `STATUS_REG.DONE = 0`.
- A write to `CONTROL_REG` with `START=1` triggers transition to START.

START State

- Latch `IMAGE_WIDTH`, `IMAGE_HEIGHT`, and `THRESHOLD`.
- Reset pixel counters and window-valid logic.
- Clear `DONE` flag.
- Enable the datapath.
- Immediately transition to `BUSY` at the next clock edge.

BUSY State

- Processing of one frame is in progress.
- `STATUS_REG.BUSY = 1`.
- `STATUS_REG.DONE = 0`.
- State remains until the last output pixel is produced (`frame_done`).

DONE State

- STATUS_REG.BUSY = 0.
- STATUS_REG.DONE = 1.
- If AUTO_RESTART = 1 $\rightarrow$ automatically transition to START.
- Else $\rightarrow$ wait for software to write START=1 again.

2.6.4 START, BUSY, and DONE Semantics

The semantics visible to software are simple:

- Writing START=1:
 - Clears DONE.
 - Sets BUSY on next cycle.
 - Begins a new processing run.
- BUSY flag:
 - High from first input pixel until last output pixel.
- DONE flag:
 - Set when the final pixel is output.
 - Cleared automatically when a new START occurs.

This design keeps software interaction straightforward:

configure $\rightarrow$ start $\rightarrow$ wait for DONE.

2.6.5 AXI4-Lite Read/Write Behavior

Write Transactions On a valid write to CONTROL_REG:

- If Bit 0 (START) is written as 1 while in IDLE or DONE:
 - The FSM transitions to START.
 - The previous DONE flag is cleared.
- Bit 1 (AUTO_RESTART) can be updated at any time.

Writes to IMAGE_WIDTH, IMAGE_HEIGHT, and THRESHOLD:

- Update the corresponding configuration registers.
- Take effect at the next START.

Read Transactions Reads from:

- CONTROL_REG: return the current START (typically 0) and AUTO_RESTART configuration bits.
- STATUS_REG: reflect real-time BUSY and DONE flags.
- IMAGE_WIDTH / IMAGE_HEIGHT / THRESHOLD: return the last programmed values.

2.6.6 Optional Interrupt Support

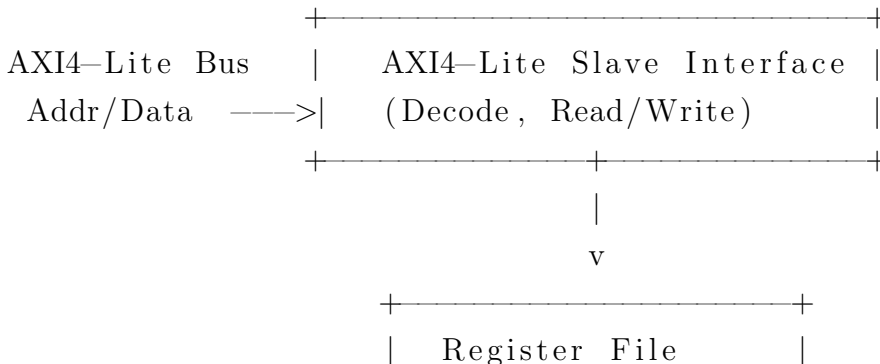
Optionally, the accelerator may expose an interrupt line `irq`:

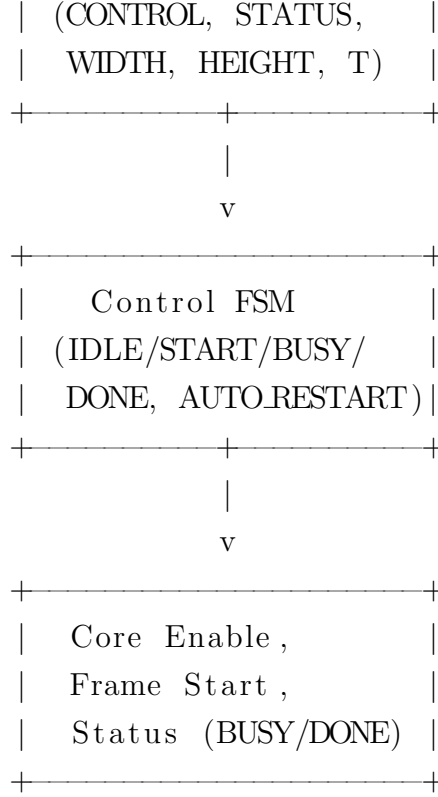
- `irq` is asserted when DONE transitions from 0 to 1.
- The RISC-V CPU can clear the interrupt by:
 - Starting a new frame (writing START), or
 - Writing to a dedicated interrupt-clear bit (if implemented).

This allows an interrupt-driven driver instead of polling STATUS_REG.

2.6.7 AXI4-Lite Control Block Diagram

Listing 7: AXI4-Lite control and status module





The AXI4-Lite module acts as the “front panel” of the accelerator, providing a clean abstraction for software while driving the underlying hardware state machine in a deterministic way.

3 Conclusion and Next Steps

3.1 Summary of Phases III & IV

This report has documented the initial stages of a hardware-software co-design project aimed at accelerating image signal processing on the BeagleV-Fire RISC-V platform.

In **Phase III**, we established a software baseline by implementing a complete ISP pipeline in C++. Through detailed profiling using `gprof` and wall-clock timing, we identified the Denoise function as the primary bottleneck, consuming over 60% of the total execution time. This provided quantitative justification for offloading this specific task to the FPGA fabric.

In **Phase IV**, we shifted focus to the theoretical architecture design. We defined a fully pipelined hardware architecture capable of one-pixel-per-cycle throughput. This included:

- Designing the **Processing Core** with line buffers and a sliding window generator to handle 3×3 spatial filtering.

- Specifying the **AXI4-Stream Interface** to ensure high-bandwidth, continuous data flow for pixel streaming.
- Defining the **AXI4-Lite Control Interface** to allow the RISC-V CPU to configure image parameters and control the execution state (START/DONE).

With the bottleneck identified and the hardware "blueprint" fully specified, the theoretical foundation is laid for the implementation phase.

3.2 Implementation Plan (Next Steps)

Moving forward, the project will transition from design and profiling to actual RTL implementation and system integration. The specific steps for the upcoming phase are as follows:

1. **RTL Implementation (Verilog Coding):** Translate the architecture defined in Phase IV into synthesizable Verilog code. This involves writing modules for the Line Buffers, Convolution Kernel, and the AXI protocol wrappers.
2. **Functional Verification (Simulation):** Develop a testbench to simulate the hardware logic. We will verify that the sliding window correctly assembles 3×3 neighborhoods and that the output pixels match the expected values from the software model. Waveform analysis will be used to confirm timing correctness.
3. **System Integration (Bitstream Generation):** Integrate the custom Denoise IP into the PolarFire SoC reference design. This involves connecting the IP to the AXI Interconnect and mapping it to the specific memory address defined in the specification. Finally, the design will be synthesized to generate the FPGA bitstream.
4. **Driver Development & Benchmarking:** Update the C++ software to replace the software-based `denoise()` function with the driver calls defined. We will then measure the new execution time and calculate the acceleration ratio compared to the Phase III baseline.

By completing these steps, we aim to demonstrate a significant performance improvement, validating the efficacy of the RISC-V + FPGA heterogeneous architecture.